

PONTIFÍCIA UNIVERSIDADE CATÓLICA DO PARANÁ
Bacharelado em Engenharia de Software

PROJETO RA3 – SISTEMA GRADLY
Aplicação Desktop em JavaFX

ARTHUR MOURA

ARTHUR DE ALMEIDA FERNANDES

DJAMES HENRIQUE

JOSUE NONALAYA

PEDRO HENRIQUE BORGES ODIA

VITOR NATAL CORDEIRO

Descrição do projeto:

O Gradly é uma plataforma web voltada para a gestão, repositório e acompanhamento de Trabalhos de Conclusão de Curso (TCC). Inspirado no modelo de fluxo de trabalho do GitHub, o sistema centraliza a comunicação, entregas de etapas, revisões e feedbacks entre alunos, professores orientadores e supervisão dos coordenadores em um único ecossistema institucional.

O Gradly estrutura funcionalidades específicas para cada papel acadêmico: os alunos podem anexar documentos e relatórios de suas entregas parciais diretamente no sistema; o orientador acessa esses arquivos para analisá-los e registrar seus feedbacks e correções de forma centralizada; e o coordenador atua na supervisão geral de todo o andamento do curso, possuindo a atribuição exclusiva de realizar a vinculação oficial entre o professor orientador e o grupo de alunos.

Classe: Instituicao

Aluno responsável

Vitor Natal Cordeiro

Descrição da Classe

A classe Instituicao representa uma instituição acadêmica cadastrada no sistema Gradly. Ela armazena informações básicas da instituição, como nome, sigla, endereço e telefone, permitindo seu gerenciamento através das funcionalidades de cadastro, consulta, edição e exclusão.

A classe implementa a interface Serializable, possibilitando que seus objetos sejam persistidos em arquivo através de serialização de objetos Java.

Atributos

1. id: identificador único da instituição.
2. nome: nome completo da instituição.
3. sigla: sigla utilizada para identificação da instituição.
4. endereco: endereço da instituição.
5. telefone: telefone para contato.

Métodos

Instituicao()

Construtor padrão da classe. Permite a criação de objetos vazios.

Instituicao(int id, String nome, String sigla, String endereco, String telefone)

Construtor parametrizado utilizado para criar um objeto já inicializado com todos os seus atributos.

Getters e Setters, permitindo consulta e definição dos atributos

Classe: InstituicaoDAO

Aluno responsável

Vitor Natal Cordeiro

Descrição da Classe

A classe `InstituicaoDAO` é responsável pela persistência dos dados da entidade `Instituição`. Ela implementa as operações de CRUD (Create, Read, Update e Delete) e realiza a leitura e gravação dos objetos em arquivo utilizando serialização.

O arquivo utilizado para armazenamento dos dados é `instituicoes.dat`.

Métodos

lerArquivo()

Método privado responsável por carregar do arquivo todas as instituições cadastradas.

Caso o arquivo não exista, retorna uma lista vazia.

gravarArquivo(List instituicoes)

Método privado responsável por persistir no arquivo a lista completa de instituições.

Também cria automaticamente a pasta de armazenamento caso ela não exista.

inserir(Instituicao instituicao)

Realiza o cadastro de uma nova instituição.

O método gera automaticamente um identificador único para a instituição, adiciona o objeto à lista existente e grava novamente os dados no arquivo.

listarTodos()

Retorna uma lista contendo todas as instituições cadastradas.

atualizar(Instituicao instituicaoAtualizada)

Localiza uma instituição através do seu identificador e atualiza seus dados com as novas informações fornecidas.

Após a alteração, a lista é gravada novamente no arquivo.

excluir(int id)

Remove uma instituição do sistema a partir do identificador informado.

Após a remoção, os dados são persistidos novamente no arquivo.

gerarNovoId(List instituicoes)

Método auxiliar responsável por gerar automaticamente um novo identificador para cada instituição cadastrada.

O novo identificador corresponde ao maior ID existente acrescido de uma unidade.

Classe: InstituicaoFormView

Aluno responsável

Vitor Natal Cordeiro

Descrição da Classe

A classe InstituicaoFormView implementa a interface gráfica responsável pelo cadastro e edição de instituições.

A tela foi desenvolvida utilizando JavaFX e apresenta os campos necessários para preenchimento dos dados da instituição.

Quando utilizada sem parâmetros, a tela opera em modo de cadastro. Quando recebe um objeto Instituicao, opera em modo de edição.

Métodos

InstituicaoFormView()

Construtor padrão utilizado para abertura da tela em modo de cadastro.

InstituicaoFormView(Instituicao instituicao)

Construtor utilizado para abertura da tela em modo de edição, carregando os dados da instituição selecionada.

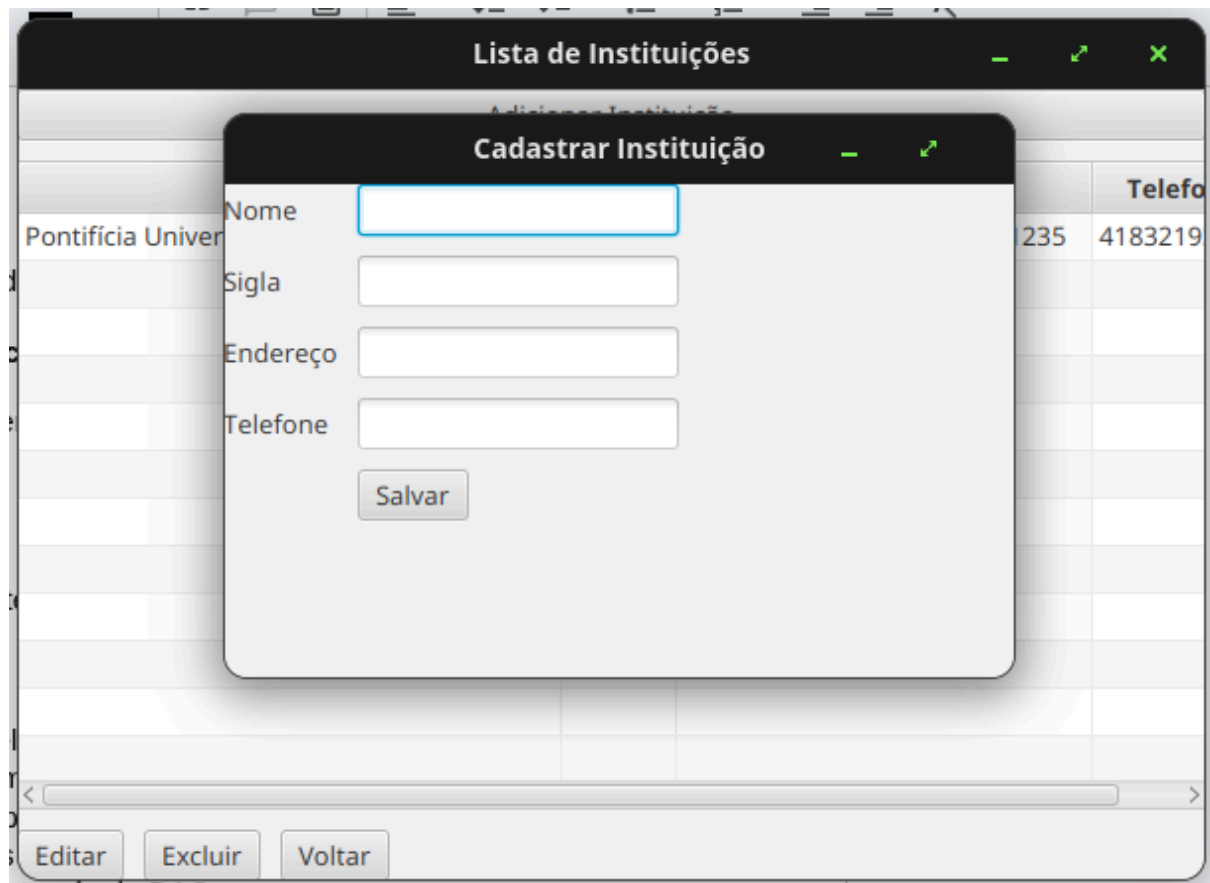
mostrar()

Responsável pela criação e exibição da interface gráfica.

Suas principais funções são:

1. Criar os componentes visuais da tela.
2. Carregar os dados da instituição em caso de edição.
3. Validar se todos os campos foram preenchidos.
4. Inserir uma nova instituição através do DAO.
5. Atualizar uma instituição existente através do DAO.
6. Exibir mensagens de erro e validação ao usuário.
7. Encerrar a janela após a operação ser concluída com sucesso.

Printscreen da tela:



Classe: InstituicaoListView

Aluno responsável:

Vitor Natal Cordeiro

Descrição da Classe

Responsável pela tela de consulta das instituições cadastradas.

Apresenta os registros em uma tabela JavaFX (TableView) e disponibiliza as funcionalidades de inserção, edição, exclusão e navegação para o menu principal.

Métodos

mostrar()

Responsável pela construção e exibição da tela de listagem.

Suas responsabilidades incluem:

1. Criar a tabela de instituições.
2. Configurar as colunas da tabela.

Lista de Instituições

Adicionar Instituição

Nome	Sigla	Endereço	Telefone
Pontifícia Universidade Católica do Paraná	PUCPR	Rua Imaculada Conceição, 1235	4183219

Editar

Excluir

Voltar

Classe: Projeto

Aluno responsável

Vitor Natal Cordeiro

Descrição da Classe

A classe Projeto representa um projeto acadêmico cadastrado no sistema Gradly. Ela armazena as informações principais de um projeto, como título, descrição, objetivo, temas abordados e estado atual.

A classe implementa a interface Serializable, permitindo que seus objetos sejam persistidos em arquivo através do mecanismo de serialização de objetos Java.

Atributos

1. id: identificador único do projeto.
2. titulo: nome ou título do projeto.
3. descricao: descrição geral do projeto.
4. objetivo: objetivo principal do projeto.
5. temas: temas ou áreas relacionadas ao projeto.
6. estado: situação atual do projeto dentro do sistema.

Métodos

Projeto()

Construtor padrão da classe. Permite a criação de objetos vazios.

Projeto(int id, String titulo, String descricao, String objetivo, String temas, String estado)

Construtor parametrizado utilizado para criar um projeto já inicializado com todos os seus atributos.

getId()

Retorna o identificador do projeto.

setId(int id)

Define o identificador do projeto.

getTitulo()

Retorna o título do projeto.

setTitulo(String titulo)

Define o título do projeto.

getDescricao()

Retorna a descrição do projeto.

setDescricao(String descricao)

Define a descrição do projeto.

getObjetivo()

Retorna o objetivo do projeto.

setObjetivo(String objetivo)

Define o objetivo do projeto.

getTemas()

Retorna os temas relacionados ao projeto.

setTemas(String temas)

Define os temas relacionados ao projeto.

getEstado()

Retorna o estado atual do projeto.

setEstado(String estado)

Define o estado atual do projeto.

Classe: ProjetoDAO

Aluno responsável

Vitor Natal Cordeiro

Descrição da Classe

A classe ProjetoDAO é responsável pelo gerenciamento da persistência da entidade Projeto. Ela implementa as operações de CRUD (Create, Read, Update e Delete), além de funcionalidades específicas para consulta e alteração do estado dos projetos.

A persistência é realizada através da serialização de objetos Java, armazenando os registros no arquivo projetos.dat.

Métodos

lerArquivo()

Método privado responsável por carregar todos os projetos armazenados no arquivo de persistência.

Caso o arquivo não exista ou esteja vazio, retorna uma lista vazia.

gravarArquivo(List projetos)

Método privado responsável por gravar a lista completa de projetos no arquivo de persistência.

Também garante a criação da pasta de armazenamento quando necessário.

inserir(Projeto projeto)

Realiza o cadastro de um novo projeto.

O método gera automaticamente um identificador único, adiciona o projeto à lista existente e grava os dados atualizados no arquivo.

listarTodos()

Retorna uma lista contendo todos os projetos cadastrados no sistema.

atualizar(Projeto projetoAtualizado)

Atualiza os dados de um projeto existente.

O projeto é localizado através do identificador e substituído pelas novas informações fornecidas.

Após a atualização, os dados são persistidos novamente no arquivo.

excluir(int id)

Remove um projeto do sistema utilizando seu identificador.

Após a exclusão, a lista atualizada é gravada novamente no arquivo.

buscarPorEstado(String estado)

Realiza uma busca de projetos com base no estado informado.

Retorna uma lista contendo todos os projetos que possuem o estado correspondente ao parâmetro recebido.

alterarEstado(int id, String novoEstado)

Altera apenas o estado de um projeto específico.

O projeto é localizado através do identificador e seu atributo estado recebe o novo valor informado.

gerarNovoId(List projetos)

Método auxiliar responsável por gerar automaticamente um identificador único para novos projetos.

O identificador gerado corresponde ao maior ID existente acrescido de uma unidade.

Classe: ProjetoFormView

Aluno responsável

Vitor Natal Cordeiro

Descrição da Classe

Implementa a interface gráfica responsável pelo cadastro e edição de projetos.

A tela foi desenvolvida utilizando JavaFX e disponibiliza os campos necessários para preenchimento e manutenção das informações do projeto.

Quando utilizada sem parâmetros, a tela opera em modo de cadastro. Quando recebe um objeto Projeto, opera em modo de edição.

Métodos

ProjetoFormView()

Construtor padrão utilizado para abertura da tela em modo de cadastro.

ProjetoFormView(Projeto projeto)

Construtor utilizado para abertura da tela em modo de edição, carregando automaticamente os dados do projeto selecionado.

mostrar()

Método responsável pela criação e exibição da interface gráfica.

Principais responsabilidades:

1. Criar os componentes visuais da tela.
2. Carregar os dados do projeto em caso de edição.
3. Validar o preenchimento obrigatório dos campos.
4. Inserir novos projetos através do ProjetoDAO.
5. Atualizar projetos existentes através do ProjetoDAO.
6. Exibir mensagens de erro e validação ao usuário.
7. Encerrar a janela após a conclusão da operação.

Classe: ProjetoListView

Aluno responsável:

Vitor Natal Cordeiro

Descrição da Classe

Responsável pela interface de consulta dos projetos cadastrados no sistema.

Exibe os registros em uma tabela JavaFX (TableView) e disponibiliza funcionalidades para cadastro, edição, exclusão e navegação para o menu principal.

Métodos

mostrar()

Responsável pela construção e exibição da tela de listagem de projetos.

Suas responsabilidades incluem:

1. Criar a tabela de projetos.
2. Configurar as colunas da tabela.
3. Carregar os dados persistidos através do ProjetoDAO.
4. Permitir o cadastro de novos projetos.
5. Permitir a edição de projetos existentes.
6. Permitir a exclusão de projetos.
7. Exibir mensagens de validação e erro ao usuário.
8. Retornar ao menu principal do sistema.

A tabela apresenta: Título, Descrição, Objetivo, Temas e Estado

Os dados exibidos são carregados diretamente da persistência implementada pela classe ProjetoDAO, garantindo que a interface reflita as informações armazenadas no sistema.

Classe: Usuario

Aluno responsável: Djames Henrique Renunza dos Santos

Descrição da Classe

A classe abstrata Usuario representa a entidade base para os usuários do sistema Gradly. Ela implementa a interface Serializable, permitindo que seus objetos sejam armazenados em arquivos. A classe contém os atributos e métodos comuns a todos os tipos de usuários do sistema, servindo como superclasse para outras entidades, como Orientador.

Atributos

- serialVersionUID: identificador utilizado no processo de serialização da classe.
- id: identificador único do usuário.
- nome: nome completo do usuário.
- email: endereço de e-mail do usuário.
- senha: senha de acesso do usuário.

Métodos

- Usuario(int id, String nome, String email, String senha): construtor responsável por inicializar os atributos do usuário.
- getId(): retorna o identificador do usuário.
- setId(int id): altera o identificador do usuário.
- getNome(): retorna o nome do usuário.
- setNome(String nome): altera o nome do usuário.
- getEmail(): retorna o e-mail do usuário.
- setEmail(String email): altera o e-mail do usuário.
- getSenha(): retorna a senha do usuário.
- setSenha(String senha): altera a senha do usuário.

Classe: Orientador

Aluno responsável: Djames Henrique Renunza dos Santos

Descrição da Classe

A classe Orientador representa um professor orientador do sistema Gradly. Ela herda os atributos e comportamentos da classe Usuario e adiciona informações específicas relacionadas à atuação acadêmica do orientador, como sua área de atuação e seu título acadêmico.

Atributos

- `areaAtuacao`: área de especialização ou atuação do orientador.
- `titulo`: titulação acadêmica do orientador, como Mestre, Doutor ou Pós-Doutor.

Métodos

- `Orientador(int id, String nome, String email, String senha, String areaAtuacao, String titulo)`: construtor responsável por inicializar os atributos da classe.
- `getAreaAtuacao()`: retorna a área de atuação do orientador.
- `setAreaAtuacao(String areaAtuacao)`: altera a área de atuação do orientador.
- `getTitulo()`: retorna o título acadêmico do orientador.
- `setTitulo(String titulo)`: altera o título acadêmico do orientador.

Classe: OrientadorDAO

Aluno responsável: Djames Henrique Renunza dos Santos

Descrição da Classe

A classe OrientadorDAO é responsável pela persistência dos dados dos orientadores. Ela realiza as operações de cadastro, consulta, atualização e exclusão (CRUD), utilizando serialização de objetos em arquivo para armazenar os dados de forma permanente.

Atributos

- `ARQUIVO`: constante que armazena o caminho do arquivo utilizado para persistência dos dados dos orientadores.

Métodos

- `criar(Orientador orientador)`: adiciona um novo orientador ao arquivo de dados.
- `ler()`: retorna a lista de orientadores cadastrados.
- `editar(Orientador orientadorAtualizado)`: atualiza os dados de um orientador existente.
- `excluir(int id)`: remove um orientador com base no ID informado.
- `gerarNovoId(ArrayList<Orientador> orientadores)`: gera automaticamente um novo identificador para o próximo cadastro.
- `lerArquivo()`: realiza a leitura do arquivo de persistência e retorna os orientadores armazenados.

- salvarArquivo(ArrayList<Orientador> orientadores): salva a lista de orientadores no arquivo de persistência.

Classe: OrientadorFormView

Aluno responsável: Djames Henrique Renunza dos Santos

Descrição da Classe

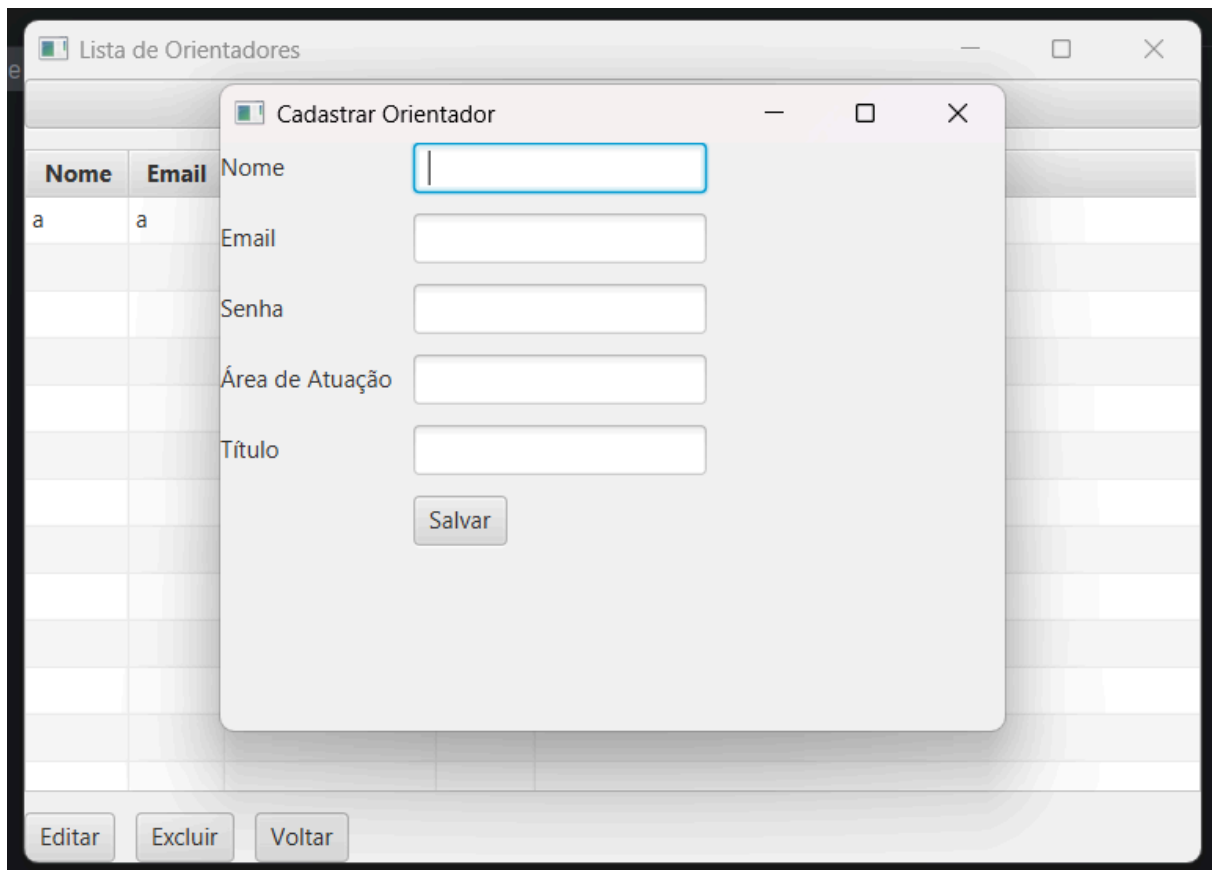
A classe OrientadorFormView é responsável pela interface gráfica de cadastro e edição de orientadores. Ela permite ao usuário informar os dados necessários para inclusão ou alteração de registros no sistema.

Atributos

- orientador: objeto utilizado para armazenar os dados do orientador que será editado.

Métodos

- OrientadorFormView(): construtor padrão utilizado para o cadastro de novos orientadores.
- OrientadorFormView(Orientador orientador): construtor utilizado para edição de um orientador existente.
- mostrar(): exibe a janela de cadastro ou edição, realiza validações dos campos e salva os dados por meio da classe DAO.



Classe: OrientadorListView

Aluno responsável: Djames Henrique Renunza dos Santos

Descrição da Classe

A classe `OrientadorListView` é responsável pela exibição da lista de orientadores cadastrados no sistema. Ela fornece funcionalidades para visualizar, adicionar, editar e excluir orientadores por meio de uma interface gráfica desenvolvida em JavaFX.

Atributos

- Não possui atributos de instância.
- Utiliza componentes gráficos locais criados dentro do método `mostrar()`.

Métodos

- `mostrar()`: exibe a janela contendo a tabela de orientadores e os botões para gerenciamento dos registros, permitindo adicionar, editar, excluir e retornar ao menu principal.

Lista de Orientadores

Adicionar Orientador

Nome	Email	Área de Atuação	Título	
a	a	a	a	
teste	teste@email.com	Física	Mestre	

Editar

Excluir

Voltar

Classe: Coordenador

Aluno responsável: DJames Henrique Renunza dos Santos

Descrição da Classe

A classe Coordenador representa um coordenador acadêmico do sistema Gradly. Ela herda os atributos e comportamentos da classe Usuario e adiciona informações específicas relacionadas ao setor acadêmico em que atua, como departamento e instituição de ensino.

Atributos

- departamento: departamento ao qual o coordenador está vinculado.
- instituicao: instituição de ensino em que o coordenador exerce suas atividades.

Métodos

- Coordenador(int id, String nome, String email, String senha, String departamento, String instituicao): construtor responsável por inicializar os atributos da classe.
- getDepartamento(): retorna o departamento do coordenador.

- setDepartamento(String departamento): altera o departamento do coordenador.
- getInstituicao(): retorna a instituição do coordenador.
- setInstituicao(String instituicao): altera a instituição do coordenador.

Classe: CoordenadorDAO

Aluno responsável: DJames Henrique Renunza dos Santos

Descrição da Classe

A classe CoordenadorDAO é responsável pela persistência dos dados dos coordenadores. Ela realiza as operações de cadastro, consulta, atualização e exclusão (CRUD), utilizando serialização de objetos em arquivo para armazenar os dados de forma permanente.

Atributos

- ARQUIVO: constante que armazena o caminho do arquivo utilizado para persistência dos dados dos coordenadores.

Métodos

- criar(Coordenador coordenador): adiciona um novo coordenador ao arquivo de dados.
- ler(): retorna a lista de coordenadores cadastrados.
- editar(Coordenador coordenadorAtualizado): atualiza os dados de um coordenador existente.
- excluir(int id): remove um coordenador com base no ID informado.
- gerarNovoId(ArrayList<Coordenador> coordenadores): gera automaticamente um novo identificador para o próximo cadastro.
- lerArquivo(): realiza a leitura do arquivo de persistência e retorna os coordenadores armazenados.
- salvarArquivo(ArrayList<Coordenador> coordenadores): salva a lista de coordenadores no arquivo de persistência.

Classe: CoordenadorFormView

Aluno responsável: DJames Henrique Renunza dos Santos

Descrição da Classe

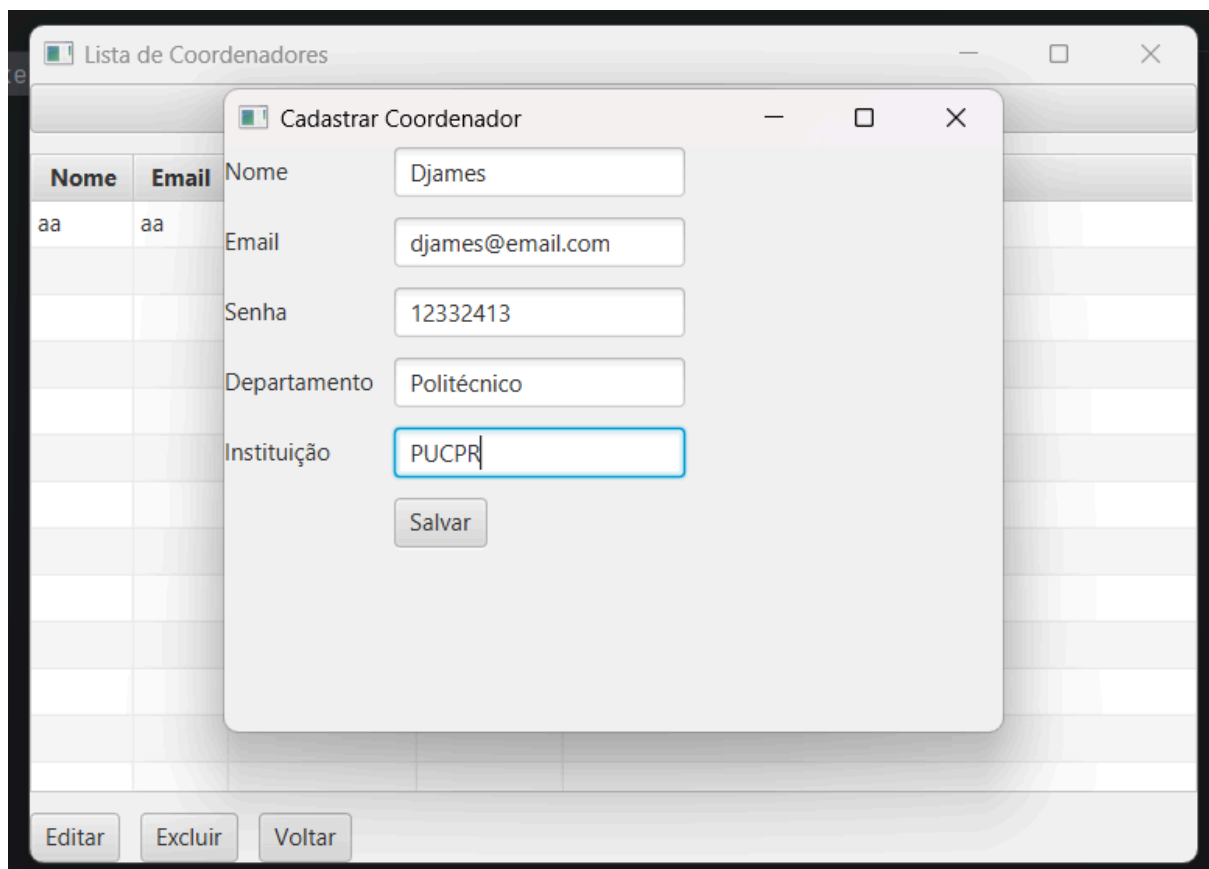
A classe CoordenadorFormView é responsável pela interface gráfica de cadastro e edição de coordenadores. Ela permite ao usuário informar os dados necessários para inclusão ou alteração de registros no sistema.

Atributos

- coordenador: objeto utilizado para armazenar os dados do coordenador que será editado.

Métodos

- CoordenadorFormView(): construtor padrão utilizado para o cadastro de novos coordenadores.
- CoordenadorFormView(Coordenador coordenador): construtor utilizado para edição de um coordenador existente.
- mostrar(): exibe a janela de cadastro ou edição, realiza validações dos campos e salva os dados por meio da classe DAO.



Classe: CoordenadorListView

Aluno responsável: Djames Henrique Renunza dos Santos

Descrição da Classe

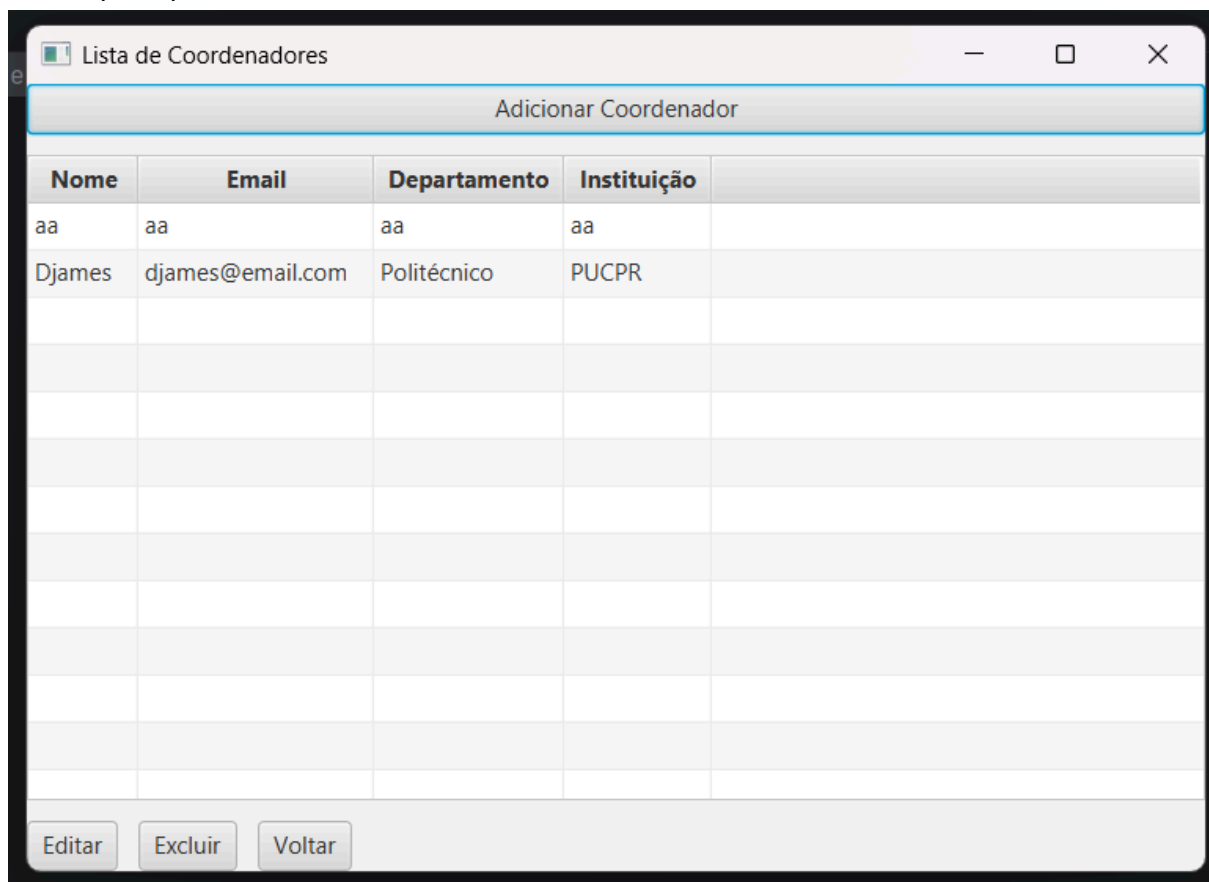
A classe `CoordenadorListView` é responsável pela exibição da lista de coordenadores cadastrados no sistema. Ela fornece funcionalidades para visualizar, adicionar, editar e excluir coordenadores por meio de uma interface gráfica desenvolvida em JavaFX.

Atributos

- Não possui atributos de instância.
- Utiliza componentes gráficos locais criados dentro do método mostrar().

Métodos

- **mostrar():** exibe a janela contendo a tabela de coordenadores e os botões para gerenciamento dos registros, permitindo adicionar, editar, excluir e retornar ao menu principal.



Classe: Documento

Aluno responsável: Pedro Henrique Borges Odia

Descrição da Classe

A classe Documento representa o modelo de domínio de um documento do projeto de TCC no sistema GRADLY. Armazena as informações de título, conteúdo, versão e data de criação, servindo como base para as operações de CRUD realizadas pelo DocumentoDAO e pela interface DocumentoView.

Atributos

id: identificador único do documento.

titulo: título do documento.

conteudo: conteúdo textual do documento.

versao: número da versão do documento.

dataCriacao: data em que o documento foi criado.

Métodos

Documento(): construtor padrão sem parâmetros.

Documento(int id, String titulo, String conteudo, int versao, LocalDate dataCriacao):

construtor com todos os atributos.

getId() / setId(int id): retorna e define o identificador do documento.

getTitulo() / setTitulo(String titulo): retorna e define o título do documento.

getConteudo() / setConteudo(String conteudo): retorna e define o conteúdo do documento.

getVersao() / setVersao(int versao): retorna e define a versão do documento.

getDataCriacao() / setDataCriacao(LocalDate dataCriacao): retorna e define a data de criação.

toString(): retorna uma representação textual do documento com título e versão.

GRADLY - Documentos

Título:

Conteúdo:

Versão:

Data:

ID	Título	Versão	Data
1	Documento 4	1	2026-06-17
2	Documento 4	2	2026-06-17

Classe: Comentario

Aluno responsável: Pedro Henrique Borges Odia

Descrição da Classe

A classe Comentario representa o modelo de domínio de um comentário de feedback do orientador no sistema GRADLY. Armazena o texto do comentário, o autor, a data de criação e o vínculo opcional com um documento, sendo utilizada nas operações de CRUD do ComentarioDAO e na interface ComentarioView.

Atributos

id: identificador único do comentário.

texto: conteúdo do comentário.

autor: nome do autor do comentário.

dataCriacao: data em que o comentário foi criado.

idDocumento: identificador do documento vinculado ao comentário.

Métodos

Comentario(): construtor padrão sem parâmetros.

Comentario(int id, String texto, String autor, LocalDate dataCriacao, int idDocumento): construtor com todos os atributos.

getId() / setId(int id): retorna e define o identificador do comentário.

getTexto() / setTexto(String texto): retorna e define o texto do comentário.

getAutor() / setAutor(String autor): retorna e define o autor do comentário.

getDataCriacao() / setDataCriacao(LocalDate dataCriacao): retorna e define a data de criação.

getIdDocumento() / setIdDocumento(int idDocumento): retorna e define o ID do documento vinculado.

GRADLY - Comentários

Texto:

Autor:

Data:

ID Documento:

ID	Texto	Autor	Data	Doc.	
1	Parabens pelo projeto	Professor	2026-06-17	1	

Classe: Administrador

Aluno responsável

Arthur Moura

Descrição da Classe

A classe Administrador representa o usuário com permissões administrativas dentro do sistema Gradly. Ela herda da classe abstrata Usuario, reaproveitando seus atributos e métodos comuns (id, nome, email e senha) e, por consequência, também é serializável, permitindo que seus objetos sejam persistidos em arquivo. Além dos dados herdados, a classe acrescenta o nível de acesso, que define o grau de permissão do administrador no sistema.

Atributos

- nivelAcesso: nível de permissão do administrador no sistema (atributo próprio da classe).
- id, nome, email, senha: dados básicos do usuário, herdados da superclasse Usuario.

Métodos

Administrador()

Construtor padrão da classe. Inicializa um administrador com valores neutros, chamando o construtor da superclasse Usuario, e permite a criação de objetos vazios para posterior preenchimento por meio dos setters.

Administrador(int id, String nome, String email, String senha, int nivelAcesso)

Construtor parametrizado utilizado para criar um administrador já inicializado com todos os seus atributos, incluindo os herdados de Usuario.

getNivelAcesso()

Retorna o nível de acesso do administrador.

setNivelAcesso(int nivelAcesso)

Define o nível de acesso do administrador.

Getters e Setters herdados de Usuario (getId, setId, getNome, setNome, getEmail, setEmail, getSenha, setSenha), permitindo a consulta e a definição dos atributos da superclasse.

Classe: AdministradorDAO

Aluno responsável

Arthur Moura

Descrição da Classe

A classe AdministradorDAO é responsável pela persistência dos dados da entidade Administrador. Ela implementa as operações de CRUD (Create, Read, Update e Delete) e realiza a leitura e a gravação dos objetos em arquivo utilizando serialização. O arquivo utilizado para armazenamento dos dados é administradores.dat.

Métodos

lerArquivo()

Método privado responsável por carregar do arquivo todos os administradores cadastrados. Caso o arquivo não exista, retorna uma lista vazia.

gravarArquivo(List administradores)

Método privado responsável por persistir no arquivo a lista completa de administradores. Também cria automaticamente a pasta de armazenamento caso ela não exista.

inserir(Administrador adm)

Realiza o cadastro de um novo administrador. O método gera automaticamente um identificador único, adiciona o objeto à lista existente e grava novamente os dados no arquivo.

listarTodos()

Retorna uma lista contendo todos os administradores cadastrados.

atualizar(Administrador admAtualizado)

Localiza um administrador através do seu identificador e atualiza seus dados com as novas informações fornecidas. Após a alteração, a lista é gravada novamente no arquivo.

excluir(int id)

Remove um administrador do sistema a partir do identificador informado. Após a remoção, os dados são persistidos novamente no arquivo.

buscarPorNivel(int nivel)

Retorna uma lista com os administradores que possuem o nível de acesso informado, permitindo a filtragem dos registros.

gerarNovoId(List administradores)

Método auxiliar responsável por gerar automaticamente um novo identificador para cada administrador cadastrado. O novo identificador corresponde ao maior ID existente acrescido de uma unidade.

Classe: AdministradorFormView

Aluno responsável

Arthur Moura

Descrição da Classe

A classe AdministradorFormView implementa a interface gráfica responsável pelo cadastro e edição de administradores. A tela foi desenvolvida utilizando JavaFX e apresenta os campos necessários para preenchimento dos dados do administrador. Quando utilizada sem parâmetros, a tela opera em modo de cadastro. Quando recebe um objeto Administrador, opera em modo de edição.

Métodos

AdministradorFormView()

Construtor padrão utilizado para abertura da tela em modo de cadastro.

AdministradorFormView(Administrador administrador)

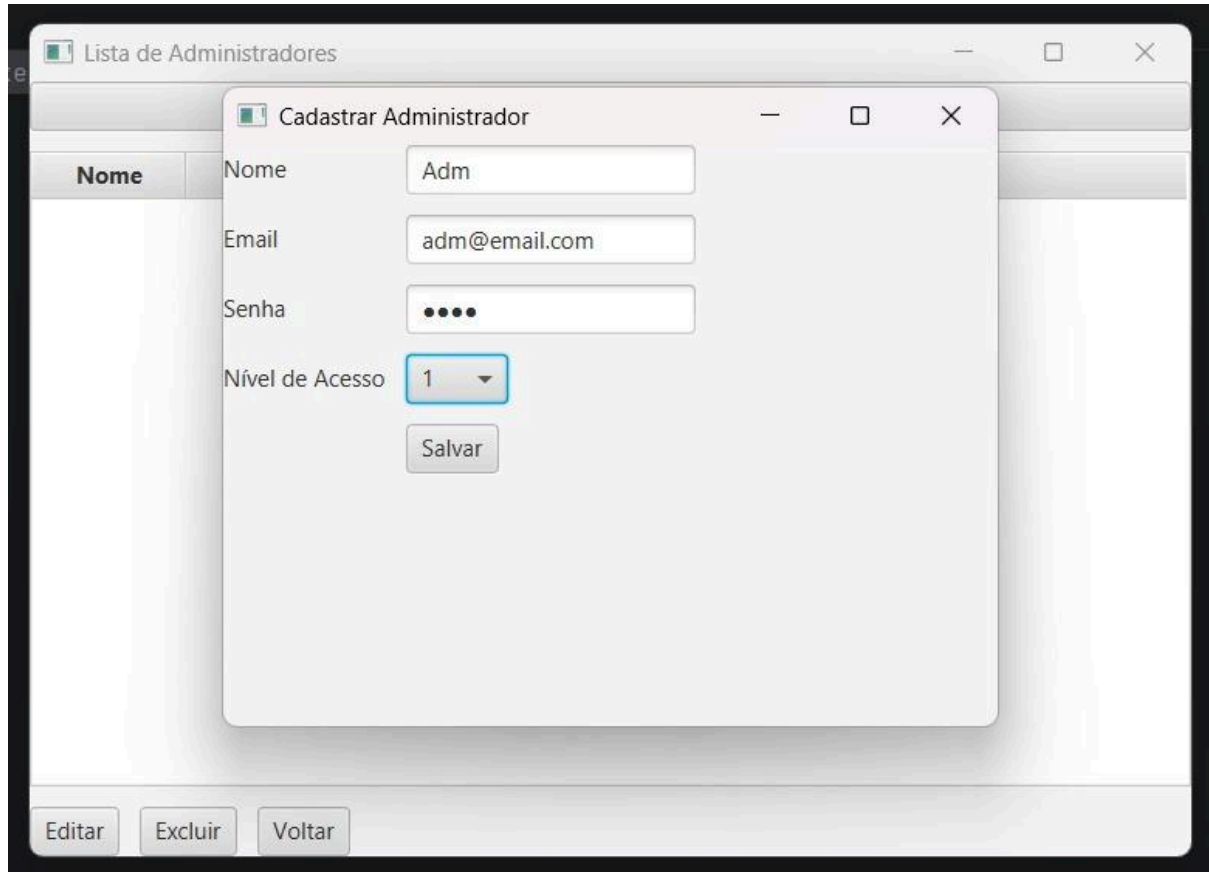
Construtor utilizado para abertura da tela em modo de edição, carregando os dados do administrador selecionado.

mostrar()

Responsável pela criação e exibição da interface gráfica. Suas principais funções são:

- Criar os componentes visuais da tela (campos de nome, email, senha e nível de acesso).
- Carregar os dados do administrador em caso de edição.
- Validar se todos os campos foram preenchidos.
- Inserir um novo administrador através do DAO.
- Atualizar um administrador existente através do DAO.
- Exibir mensagens de erro e validação ao usuário.
- Encerrar a janela após a operação ser concluída com sucesso.

Printscreen da tela:



Classe: AdministradorListView

Aluno responsável

Arthur Moura

Descrição da Classe

Responsável pela tela de consulta dos administradores cadastrados. Apresenta os registros em uma tabela JavaFX (TableView) e disponibiliza as funcionalidades de inserção, edição, exclusão e navegação para o menu principal.

Métodos

mostrar()

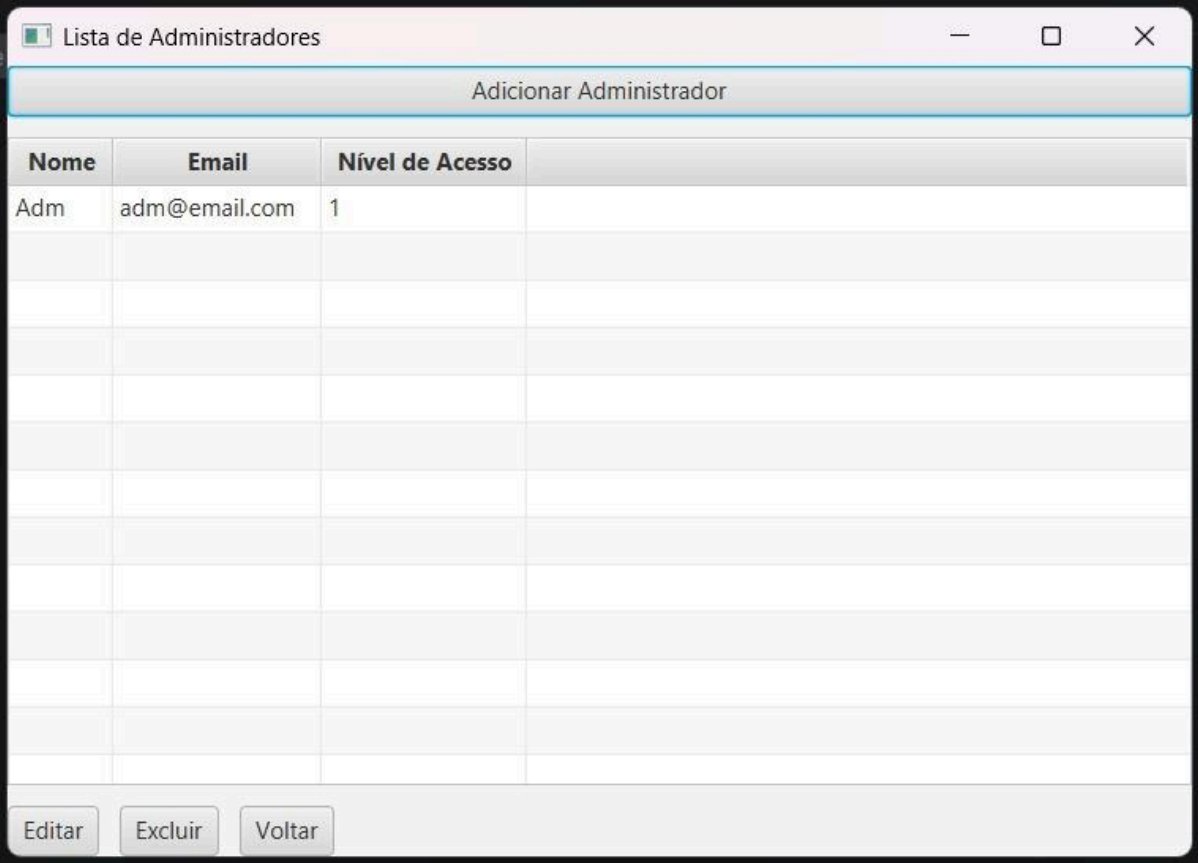
Responsável pela construção e exibição da tela de listagem. Suas responsabilidades incluem:

- Criar a tabela de administradores.
- Configurar as colunas da tabela.
- Carregar os dados persistidos através do DAO.
- Permitir o cadastro de novos administradores.
- Permitir a edição de administradores existentes.
- Permitir a exclusão de administradores.
- Exibir mensagens de erro e validação.
- Retornar ao menu principal do sistema.

A tabela apresenta: Nome, Email e Nível de Acesso.

Os dados exibidos são carregados diretamente da persistência implementada pela classe AdministradorDAO.

Printscreen da tela:



Nome	Email	Nível de Acesso
Adm	adm@email.com	1

Classe: Banca

Aluno responsável

Arthur Moura

Descrição da Classe

A classe Banca representa a banca de avaliação de um projeto de TCC no sistema Gradly. Ela armazena as informações da defesa do trabalho, como data, sala, nota e parecer, e mantém um vínculo com o projeto avaliado por meio do atributo projetoid, estabelecendo o relacionamento entre a banca e o projeto correspondente.

A classe implementa a interface Serializable, permitindo que seus objetos sejam persistidos em arquivo através do mecanismo de serialização de objetos Java.

Atributos

- id: identificador único da banca.
- data: data de realização da banca de avaliação.
- sala: local (sala) em que a banca será realizada.
- nota: nota atribuída ao projeto pela banca.

- parecer: texto com o parecer ou as observações da banca sobre o projeto.
- projetold: identificador do projeto ao qual a banca está vinculada, estabelecendo o relacionamento entre as entidades.

Métodos

Banca()

Construtor padrão da classe. Permite a criação de objetos vazios.

Banca(int id, LocalDate data, String sala, double nota, String parecer, int projetold)

Construtor parametrizado utilizado para criar uma banca já inicializada com todos os seus atributos.

getId() / setId(int id)

Retornam e definem o identificador da banca.

getData() / setData(LocalDate data)

Retornam e definem a data da banca.

getSala() / setSala(String sala)

Retornam e definem a sala da banca.

getNota() / setNota(double nota)

Retornam e definem a nota atribuída pela banca.

getParecer() / setParecer(String parecer)

Retornam e definem o parecer da banca.

getProjetold() / setProjetold(int projetold)

Retornam e definem o identificador do projeto vinculado à banca.

Classe: BancaDAO

Aluno responsável

Arthur Moura

Descrição da Classe

A classe BancaDAO é responsável pela persistência dos dados da entidade Banca. Ela implementa as operações de CRUD (Create, Read, Update e Delete) e realiza a leitura e a gravação dos objetos em arquivo utilizando serialização. O arquivo utilizado para armazenamento dos dados é bancas.dat.

Métodos

lerArquivo()

Método privado responsável por carregar do arquivo todas as bancas cadastradas. Caso o arquivo não exista, retorna uma lista vazia.

gravarArquivo(List bancas)

Método privado responsável por persistir no arquivo a lista completa de bancas. Também cria automaticamente a pasta de armazenamento caso ela não exista.

inserir(Banca banca)

Realiza o cadastro de uma nova banca. O método gera automaticamente um identificador único, adiciona o objeto à lista existente e grava novamente os dados no arquivo.

listarTodos()

Retorna uma lista contendo todas as bancas cadastradas.

atualizar(Banca bancaAtualizada)

Localiza uma banca através do seu identificador e atualiza seus dados com as novas informações fornecidas. Após a alteração, a lista é gravada novamente no arquivo.

excluir(int id)

Remove uma banca do sistema a partir do identificador informado. Após a remoção, os dados são persistidos novamente no arquivo.

buscarPorProjeto(int projetoid)

Retorna uma lista com as bancas vinculadas ao projeto informado, permitindo a filtragem dos registros pelo projeto relacionado.

gerarNovoid(List bancas)

Método auxiliar responsável por gerar automaticamente um novo identificador para cada banca cadastrada. O novo identificador corresponde ao maior ID existente acrescido de uma unidade.

Classe: BancaFormView

Aluno responsável

Arthur Moura

Descrição da Classe

A classe BancaFormView implementa a interface gráfica responsável pelo cadastro e edição de bancas. A tela foi desenvolvida utilizando JavaFX e apresenta os campos necessários para preenchimento dos dados da banca, incluindo um seletor de data no formato brasileiro (DD/MM/AAAA) e um campo de seleção do projeto ao qual a banca será vinculada. Quando utilizada sem parâmetros, a tela opera em modo de cadastro. Quando recebe um objeto Banca, opera em modo de edição.

Métodos

BancaFormView()

Construtor padrão utilizado para abertura da tela em modo de cadastro.

BancaFormView(Banca banca)

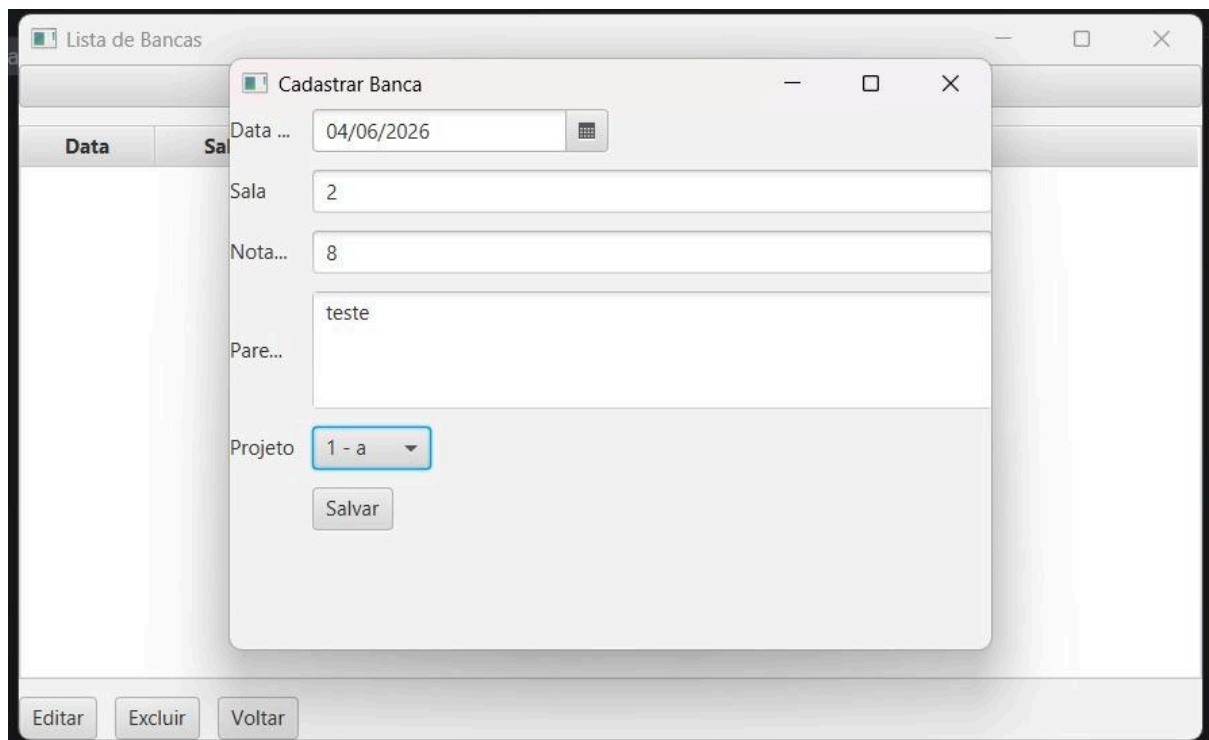
Construtor utilizado para abertura da tela em modo de edição, carregando os dados da banca selecionada.

mostrar()

Responsável pela criação e exibição da interface gráfica. Suas principais funções são:

- Criar os componentes visuais da tela (data, sala, nota, parecer e seleção de projeto).
- Listar os projetos existentes no campo de seleção, por meio do ProjetoDAO, estabelecendo o relacionamento entre banca e projeto.
- Carregar os dados da banca em caso de edição.
- Validar se todos os campos foram preenchidos.
- Tratar o campo de nota como valor numérico e validar a faixa permitida (0 a 10).
- Apresentar e interpretar a data no formato brasileiro (DD/MM/AAAA).
- Inserir uma nova banca através do DAO.
- Atualizar uma banca existente através do DAO.
- Exibir mensagens de erro e validação ao usuário.
- Encerrar a janela após a operação ser concluída com sucesso.

Printscreen da tela:



Classe: BancaListView

Aluno responsável

Arthur Moura

Descrição da Classe

Responsável pela tela de consulta das bancas cadastradas. Apresenta os registros em uma tabela JavaFX (TableView) e disponibiliza as funcionalidades de inserção, edição, exclusão e navegação para o menu principal.

Métodos

mostrar()

Responsável pela construção e exibição da tela de listagem. Suas responsabilidades incluem:

- Criar a tabela de bancas.
- Configurar as colunas da tabela, exibindo a data no formato brasileiro e o título do projeto relacionado.
- Carregar os dados persistidos através do DAO.
- Permitir o cadastro de novas bancas.
- Permitir a edição de bancas existentes.
- Permitir a exclusão de bancas.
- Exibir mensagens de erro e validação.
- Retornar ao menu principal do sistema.

A tabela apresenta: Data, Sala, Nota, Parecer e Projeto.

Os dados exibidos são carregados diretamente da persistência implementada pela classe BancaDAO. A coluna Projeto utiliza o ProjetoDAO para exibir o título do projeto vinculado a cada banca.

Printscreen da tela:

 Lista de Bancas

Adicionar Banca

Data	Sala	Nota	Parecer	Projeto
04/06/2026	2	8.0	teste	a

Classe: Tarefa

Aluno responsável: Arthur Fernandes

Descrição da Classe

A classe Tarefa representa uma atividade que deve ser realizada dentro do sistema Gradly. Ela armazena informações relacionadas à descrição da tarefa, seu estado atual e as datas de início e término, permitindo o acompanhamento do progresso das atividades.

Atributos

- id: identificador único da tarefa.
- descricao: descrição da atividade a ser executada.
- estado: situação atual da tarefa, podendo assumir valores como "A Fazer", "Fazendo" ou "Concluída".
- dataInicio: data prevista ou efetiva de início da tarefa.
- dataFim: data prevista ou efetiva de término da tarefa.

Métodos

- Tarefa(): construtor padrão da classe.
- Tarefa(int id, String descricao, String estado, LocalDate dataInicio, LocalDate dataFim): construtor responsável por inicializar todos os atributos da tarefa.
- getId(): retorna o identificador da tarefa.
- setId(int id): altera o identificador da tarefa.
- getDescricao(): retorna a descrição da tarefa.
- setDescricao(String descricao): altera a descrição da tarefa.
- getEstado(): retorna o estado atual da tarefa.
- setEstado(String estado): altera o estado da tarefa.
- getDataInicio(): retorna a data de início da tarefa.
- setDataInicio(LocalDate dataInicio): altera a data de início da tarefa.
- getDataFim(): retorna a data de término da tarefa.
- setDataFim(LocalDate dataFim): altera a data de término da tarefa.

Classe: TarefaDAO

Aluno responsável: Arthur Fernandes

Descrição da Classe

A classe TarefaDAO é responsável pela persistência dos dados das tarefas. Ela realiza operações de cadastro, consulta, atualização, exclusão e filtragem, utilizando serialização de objetos para armazenar as informações em arquivo.

Atributos

- ARQUIVO: constante que armazena o caminho do arquivo utilizado para persistência dos dados das tarefas.

Métodos

- lerArquivo(): realiza a leitura do arquivo de persistência e retorna a lista de tarefas armazenadas.
- gravarArquivo(List<Tarefa> tarefa): salva a lista de tarefas no arquivo de persistência.
- inserir(Tarefa tarefa): adiciona uma nova tarefa ao sistema.
- listarTodos(): retorna todas as tarefas cadastradas.
- atualizar(Tarefa tarefaAtualizada): atualiza os dados de uma tarefa existente.
- excluir(int id): remove uma tarefa com base no identificador informado.
- buscarPorEstado(String estado): retorna as tarefas que possuem o estado informado.
- alterarEstado(int id, String novoEstado): altera o estado de uma tarefa específica.
- gerarNovoId(List<Tarefa> tarefas): gera automaticamente um novo identificador para o próximo cadastro.

Classe: TarefaFormView

Aluno responsável: Arthur Fernandes

Descrição da Classe

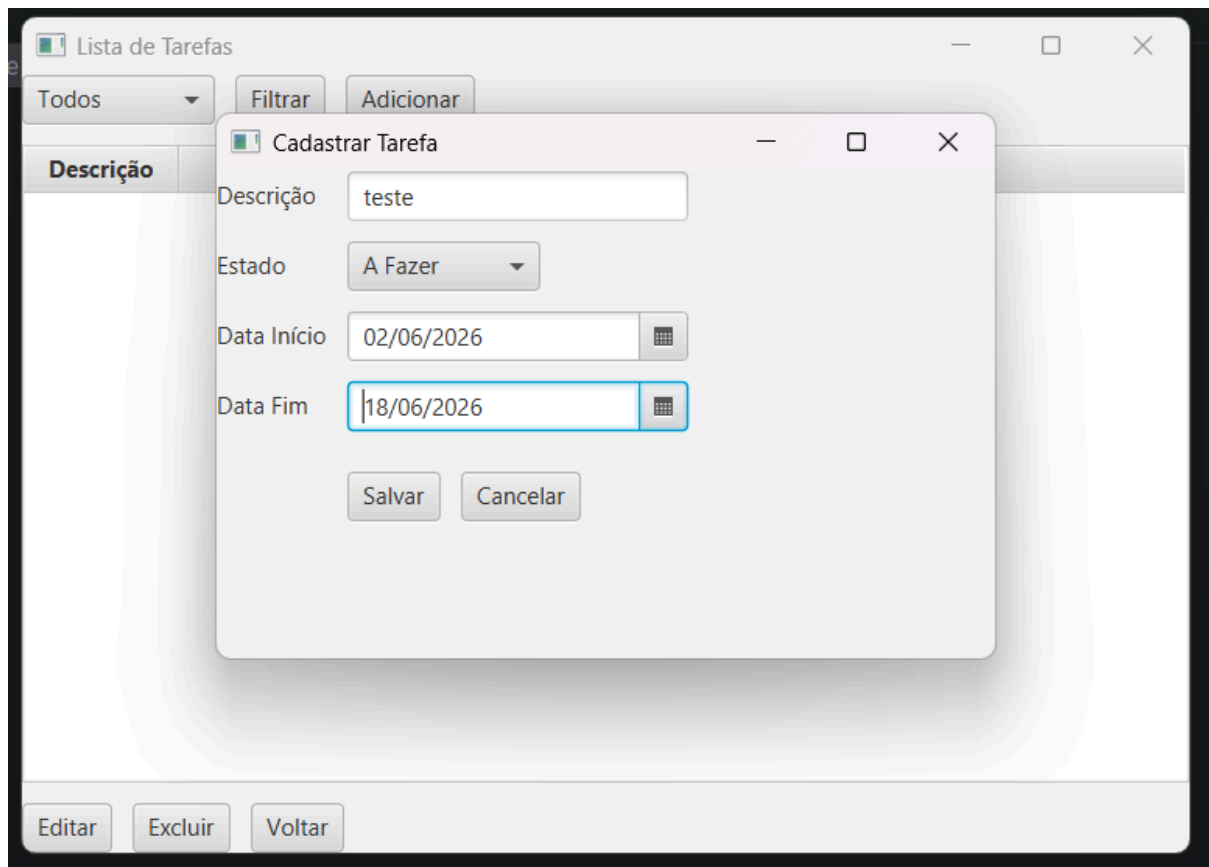
A classe TarefaFormView é responsável pela interface gráfica de cadastro e edição de tarefas. Ela permite ao usuário informar a descrição, o estado e as datas da tarefa, realizando o armazenamento ou atualização das informações por meio da camada DAO.

Atributos

- tarefa: objeto utilizado para armazenar a tarefa que será editada.

Métodos

- TarefaFormView(): construtor padrão utilizado para cadastro de novas tarefas.
- TarefaFormView(Tarefa tarefa): construtor utilizado para edição de uma tarefa existente.
- mostrar(): exibe a janela de cadastro ou edição, permitindo salvar ou cancelar a operação.



Classe: TarefaListView

Aluno responsável: Arthur Fernandes

Descrição da Classe

A classe TarefaListView é responsável pela exibição e gerenciamento das tarefas cadastradas no sistema. Ela disponibiliza recursos para visualização, filtragem por estado, inclusão, edição e exclusão de tarefas através de uma interface gráfica desenvolvida em JavaFX.

Atributos

- Não possui atributos de instância.
- Utiliza componentes gráficos locais criados dentro do método mostrar().

Métodos

- mostrar(): exibe a janela contendo a tabela de tarefas, filtros por estado e botões para adicionar, editar, excluir tarefas e retornar ao menu principal.

Lista de Tarefas

Todos

Filtrar

Adicionar

Descrição	Estado
teste	A Fazer

Editar

Excluir

Voltar

Classe: Referencia

Aluno responsável: Arthur Fernandes

Descrição da Classe

A classe Referencia representa uma referência bibliográfica cadastrada no sistema Gradly. Ela armazena informações relacionadas ao título da obra, autor, ano de publicação e tipo de material, auxiliando no gerenciamento das fontes utilizadas nos projetos acadêmicos.

Atributos

- id: identificador único da referência.
- título: título da obra ou material referenciado.
- autor: nome do autor da referência.
- ano: ano de publicação da referência.
- tipo: categoria da referência, como livro, artigo ou site.

Métodos

- Referencia(): construtor padrão da classe.
- Referencia(int id, String titulo, String autor, int ano, String tipo): construtor responsável por inicializar todos os atributos da referência.
- getId(): retorna o identificador da referência.
- setId(int id): altera o identificador da referência.
- getTitulo(): retorna o título da referência.
- setTitulo(String titulo): altera o título da referência.
- getAutor(): retorna o autor da referência.
- setAutor(String autor): altera o autor da referência.
- getAno(): retorna o ano de publicação da referência.
- setAno(int ano): altera o ano de publicação da referência.
- getTipo(): retorna o tipo da referência.
- setTipo(String tipo): altera o tipo da referência.

Classe: ReferenciaDAO

Aluno responsável: Arthur Fernandes

Descrição da Classe

A classe ReferenciaDAO é responsável pela persistência dos dados das referências bibliográficas. Ela realiza operações de cadastro, consulta, atualização, exclusão e filtragem, utilizando serialização de objetos para armazenar as informações em arquivo.

Atributos

- ARQUIVO: constante que armazena o caminho do arquivo utilizado para persistência dos dados das referências.

Métodos

- lerArquivo(): realiza a leitura do arquivo de persistência e retorna a lista de referências armazenadas.
- gravarArquivo(List<Referencia> referencia): salva a lista de referências no arquivo de persistência.
- inserir(Referencia referencia): adiciona uma nova referência ao sistema.
- listarTodos(): retorna todas as referências cadastradas.
- atualizar(Referencia tarefaAtualizada): atualiza os dados de uma referência existente.
- excluir(int id): remove uma referência com base no identificador informado.
- buscarPorAutor(String autor): retorna as referências associadas ao autor informado.
- buscarPorTipo(String tipo): retorna as referências que pertencem ao tipo informado.
- gerarNovoId(List<Referencia> referencias): gera automaticamente um novo identificador para o próximo cadastro.

Classe: ReferenciaFormView

Aluno responsável: Arthur Fernandes

Descrição da Classe

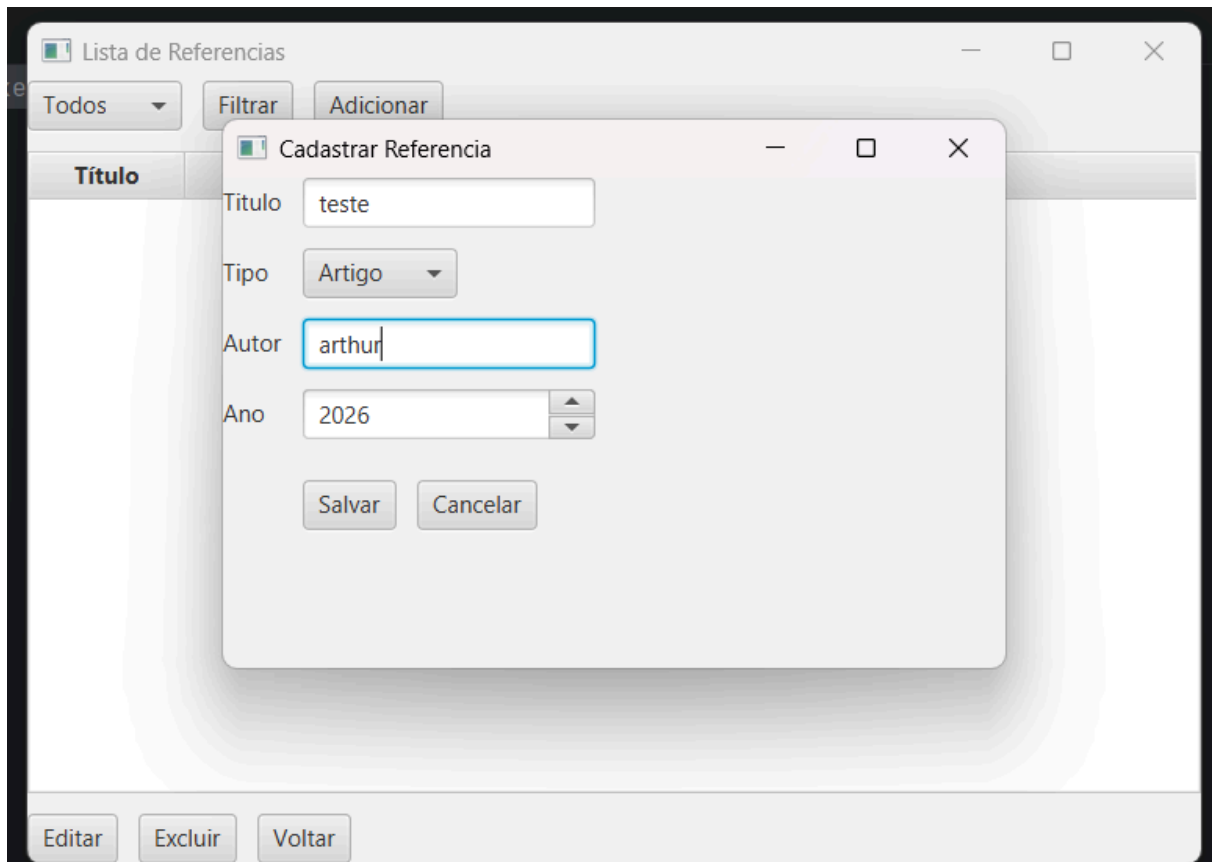
A classe ReferenciaFormView é responsável pela interface gráfica de cadastro e edição de referências bibliográficas. Ela permite ao usuário informar os dados da obra, como título, autor, ano e tipo, realizando o armazenamento ou atualização das informações por meio da camada DAO.

Atributos

- referencia: objeto utilizado para armazenar a referência que será editada.

Métodos

- ReferenciaFormView(): construtor padrão utilizado para cadastro de novas referências.
- ReferenciaFormView(Referencia referencia): construtor utilizado para edição de uma referência existente.
- mostrar(): exibe a janela de cadastro ou edição, permitindo salvar ou cancelar a operação.



Classe: ReferenciaListView

Aluno responsável: Arthur Fernandes

Descrição da Classe

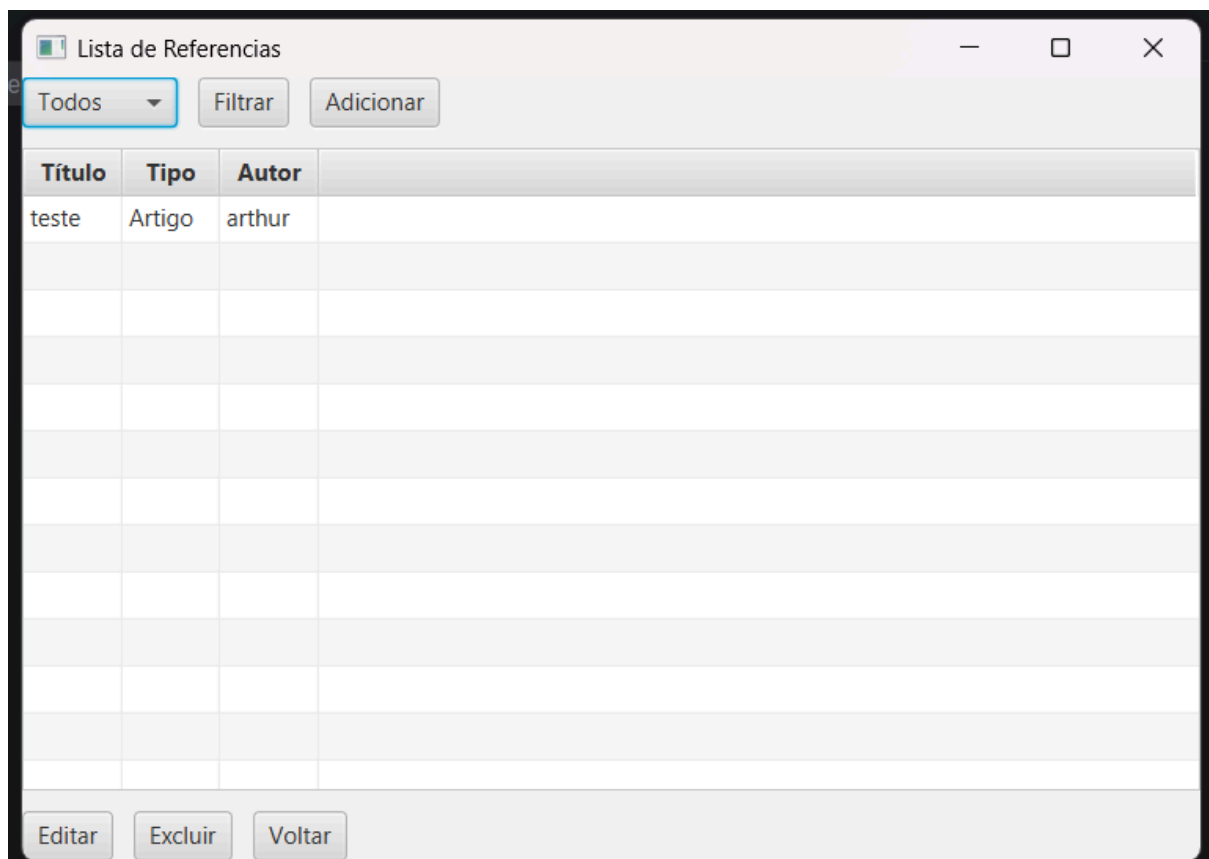
A classe `ReferenciaListView` é responsável pela exibição e gerenciamento das referências bibliográficas cadastradas no sistema. Ela disponibiliza recursos para visualização, filtragem por tipo, inclusão, edição e exclusão de referências através de uma interface gráfica desenvolvida em JavaFX.

Atributos

- Não possui atributos de instância.
- Utiliza componentes gráficos locais criados dentro do método mostrar().

Métodos

- `mostrar()`: exibe a janela contendo a tabela de referências, filtros por tipo e botões para adicionar, editar, excluir referências e retornar ao menu principal.



Classe: Aluno

Aluno responsável - Josue Nonalaya

Descrição da Classe

A classe Aluno representa a entidade aluno do nosso sistema. Ela é utilizada para armazenar os dados de um aluno, como matrícula, nome, curso, e-mail e senha. A classe implementa a interface Serializable, permitindo que seus objetos sejam gravados e recuperados de arquivos.

Atributos:

- id: Identificador único do aluno
- matricula: Número de matrícula
- nome: Nome completo do aluno
- curso: Curso do aluno
- email: Email do aluno
- senha: Senha do aluno

Métodos:

Aluno()

- Construtor padrão utilizado para criar um objeto vazio.

Aluno(int matricula, String curso)

- Construtor que inicializa matrícula e curso.

getId()

- Retorna o identificador do aluno.

setId(int id)

- Define o identificador do aluno.

getMatricula()

- Retorna a matrícula.

setMatricula(int matricula)

- Define a matrícula.

getCurso()

- Retorna o curso.

setCurso(String curso)

- Define o curso.

getNome()

- Retorna o nome.

setNome(String nome)

- Define o nome.

getEmail()

- Retorna o e-mail.

setEmail(String email)

- Define o e-mail.

getSenha()

- Retorna a senha.

setSenha(String senha)

- Define a senha.

Classe: AlunoDAO

Aluno responsável - Josue Nonalaya

Descrição da Classe

A classe AlunoDAO é responsável pela persistência dos dados dos alunos. Ela implementa as operações CRUD (Create, Read, Update e Delete), utilizando serialização de objetos para armazenar os dados em arquivo.

Métodos

lerArquivo()

- Realiza a leitura do arquivo de alunos e retorna uma lista contendo os registros armazenados.

gravarArquivo(List<Aluno> alunos)

- Grava a lista de alunos no arquivo de armazenamento.

inserir(Aluno aluno)

- Adiciona um novo aluno ao sistema e gera automaticamente um identificador único.

listarTodos()

- Retorna todos os alunos cadastrados.

atualizar(Aluno alunoAtualizado)

- Atualiza os dados de um aluno existente.

excluir(int id)

- Remove um aluno com base no identificador informado.

gerarNovoId(List<Aluno> alunos)

- Calcula o próximo identificador disponível para um novo aluno.

Classe: AlunoListView

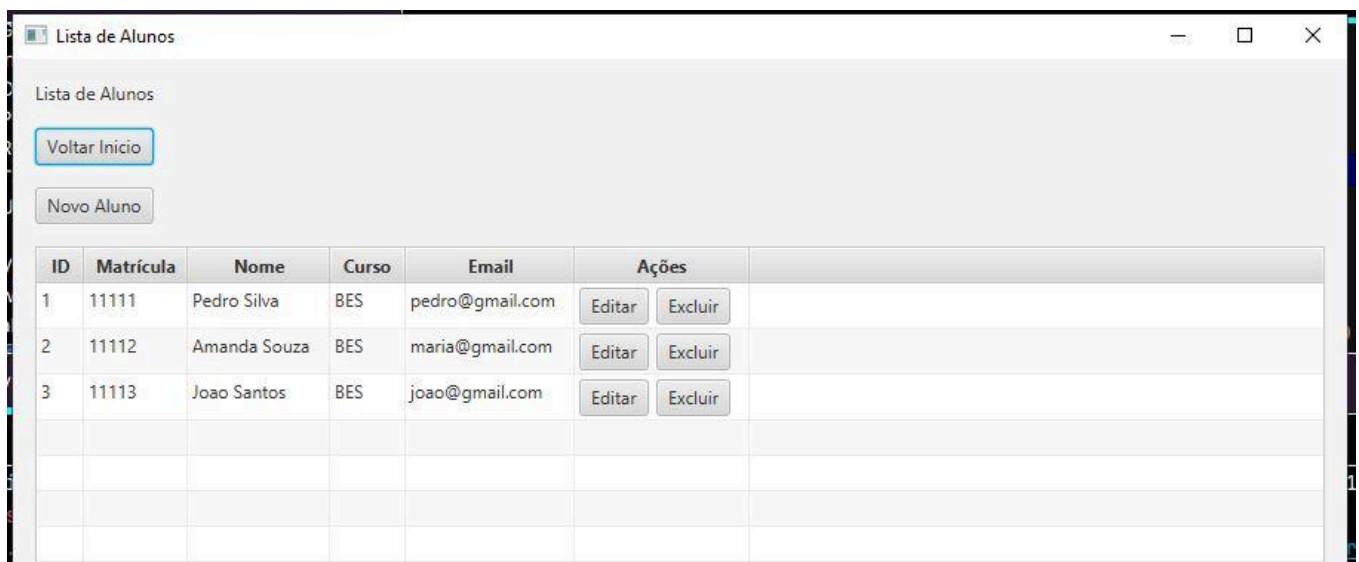
Aluno responsável - Josue Nonalaya

Descrição da Classe

A classe AlunoListView implementa a tela de listagem de alunos. Ela exibe todos os registros cadastrados em uma tabela e disponibiliza funcionalidades para inclusão, edição e exclusão de alunos.

Componentes

- Label "Lista de Alunos": Título da tela
- Button "Novo Aluno": Abre o formulário de cadastro
- Button "Voltar Início": Retorna ao menu principal
- TableView: Exibe todos os alunos cadastrados
- Button "Editar": Abre a tela de edição
- Button "Excluir": Remove o aluno selecionado



print da tela AlunosListView.java

Métodos

mostrar()

- Monta a interface gráfica da tela de listagem, configura a tabela e exibe a janela.

carregarTabela(tableView<Aluno> tabela, AlunoDAO dao)

- Atualiza os dados exibidos na tabela a partir do arquivo de armazenamento.

Classe: AlunoCreateView

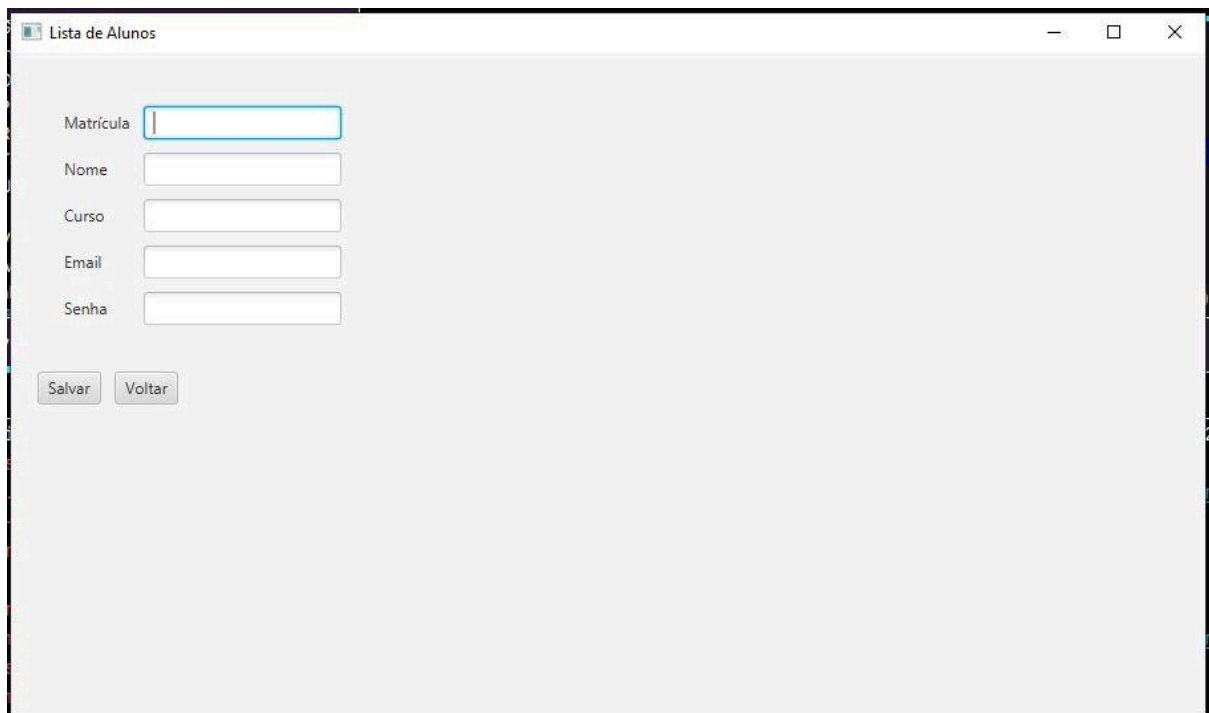
Aluno responsável - Josue Nonalaya

Descrição da Classe

A classe AlunoCreateView implementa a tela de cadastro de alunos. Ela permite ao usuário informar os dados necessários para criar um novo registro.

Componentes:

- Campo Matrícula: Informa a matrícula
- Campo Nome: Informa o nome
- Campo Curso: Informa o curso
- Campo Email: Informa o e-mail
- Campo Senha: Informa a senha
- Botão Salvar: Cadastra o aluno
- Botão Voltar: Retorna para a listagem



A imagem mostra uma janela de software com o título "Lista de Alunos". Dentro da janela, há uma interface de usuário com cinco campos de texto rotulados "Matrícula", "Nome", "Curso", "Email" e "Senha", dispostos verticalmente. Abaixo desses campos, há dois botões: "Salvar" e "Voltar".

print da tela AlunoCreateView.java

Métodos

AlunoCreateView(Stage stage)

- Construtor que recebe a janela onde a tela será exibida.

mostrar()

- Monta a interface gráfica e realiza o cadastro do aluno quando o botão Salvar é acionado.

Classe: AlunoEditView

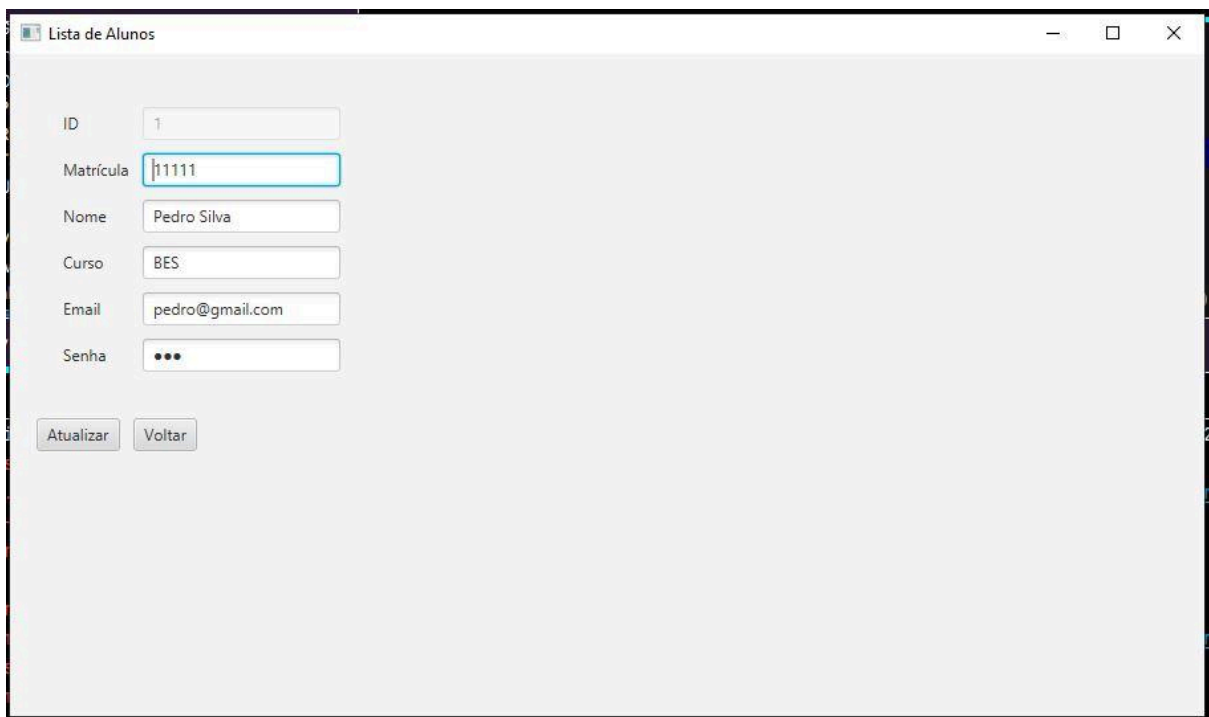
Aluno responsável - Josue Nonalaya

Descrição da Classe

A classe AlunoEditView implementa a tela de edição de alunos. Ela carrega os dados existentes do aluno e permite sua alteração.

Componentes:

- Campo ID: Exibe o identificador do aluno (somente leitura)
- Campo Matrícula: Permite alterar a matrícula
- Campo Nome: Permite alterar o nome
- Campo Curso: Permite alterar o curso
- Campo Email: Permite alterar o e-mail
- Campo Senha: Permite alterar a senha
- Botão Atualizar: Salva as alterações
- Botão Voltar: Retorna para a listagem



A imagem mostra uma janela de software com o título "Lista de Alunos". Dentro da janela, há uma interface de usuário com os seguintes elementos:

- Um campo de texto rotulado "ID" com o valor "1".
- Um campo de texto rotulado "Matrícula" com o valor "11111".
- Um campo de texto rotulado "Nome" com o valor "Pedro Silva".
- Um campo de texto rotulado "Curso" com o valor "BES".
- Um campo de texto rotulado "Email" com o valor "pedro@gmail.com".
- Um campo de texto rotulado "Senha" com caracteres mascarados por pontos.
- Dois botões de ação na base da interface: "Atualizar" e "Voltar".

print da tela AlunosEditView.java

Métodos

AlunoEditView(Stage stage, Aluno aluno)

- Construtor que recebe a janela e o aluno a ser editado.

mostrar()

- Monta a interface gráfica, carrega os dados do aluno selecionado e realiza a atualização dos dados quando solicitado.

Classe: Grupo

Aluno responsável - Josue Nonalaya

Descrição da Classe

A classe Grupo representa a entidade grupo do sistema. Ela é utilizada para armazenar informações relacionadas aos grupos de trabalho do TCC cadastrados no sistema, incluindo nome, descrição e data de criação. Esta classe também implementa a interface Serializable.

Atributos:

- id: Identificador único do grupo
- nome: Nome do grupo
- descricao: Descrição do grupo
- datacriacao: Data de criação do grupo
- senha: Senha do aluno

Métodos:

Grupo()

- Construtor padrão utilizado para criar um objeto vazio.

Grupo(String nome, String descricao, LocalDate datacriacao)

- Construtor que inicializa os principais atributos do grupo.

getId()

- Retorna o identificador do grupo.

setId(int id)

- Define o identificador do grupo.

getNome()

- Retorna o nome do grupo.

setNome(String nome)

- Define o nome do grupo.

getDescricao()

- Retorna a descrição do grupo.

setDescricao(String descricao)

- Define a descrição do grupo.

getDatacriacao()

- Retorna a data de criação do grupo.

setDatacriacao(LocalDate datacriacao)

- Define a data de criação do grupo.

Classe: GrupoDAO

Aluno responsável - Josue Nonalaya

Descrição da Classe

A classe GrupoDAO é responsável pela persistência dos dados dos grupos. Ela implementa as operações de cadastro, consulta, atualização e remoção de grupos utilizando serialização de objetos e armazenamento em arquivo.

Métodos:

lerArquivo()

- Realiza a leitura do arquivo de grupos e retorna uma lista contendo todos os registros armazenados.

gravarArquivo(List<Grupo> grupos)

- Grava a lista de grupos no arquivo de armazenamento.

inserir(Gruo grupo)

- Adiciona um novo grupo ao sistema e gera automaticamente um identificador único.
-

listarTodos()

- Retorna todos os grupos cadastrados.

atualizar(Gruo grupoAtualizado)

- Atualiza os dados de um grupo existente.

excluir(int id)

- Remove um grupo do sistema com base no identificador informado.

gerarNovold(List<Grupo> grupos)

- Calcula e retorna o próximo identificador disponível para cadastro.

Classe: GrupoListView

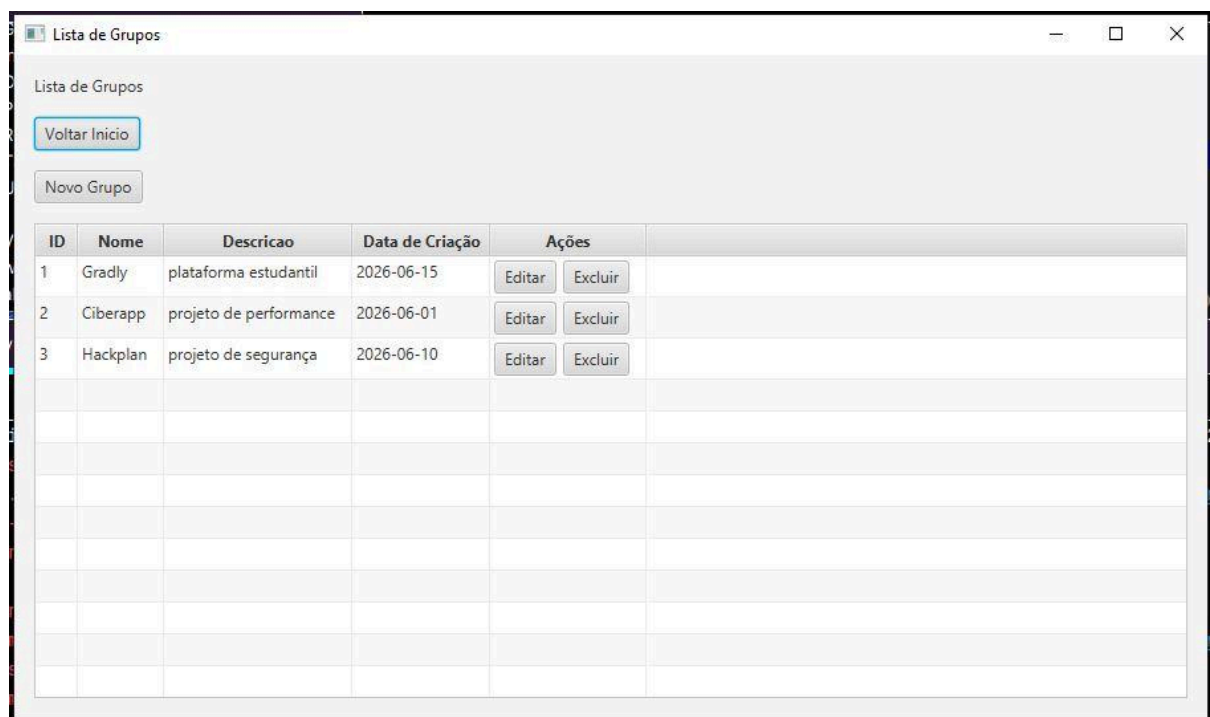
Aluno responsável - Josue Nonalaya

Descrição da Classe

A classe GrupoListView implementa a tela de listagem de todos os grupos. Na mesma são exibidos todos os grupos cadastrados do sistema, o usuário pode realizar operações de cadastro, edição e exclusão de grupos.

Componentes:

- Label "Lista de Grupos": Exibe o título da tela
- Botão "Novo Grupo": Abre a tela de cadastro de grupo
- Botão "Voltar Início": Retorna ao menu principal
- TableView: Exibe todos os grupos cadastrados
- Botão "Editar": Abre a tela de edição
- Botão "Excluir": Remove o grupo selecionado



print da tela GrupoListView.java

Métodos

mostrar()

- Monta toda a interface gráfica da tela de listagem, configura a tabela e exibe os grupos cadastrados.

carregarTabela(tableView<Grupo> grupo, GrupoDAO dao)

- Atualiza os dados exibidos na tabela consultando os registros armazenados.

Classe: GrupoCreateView

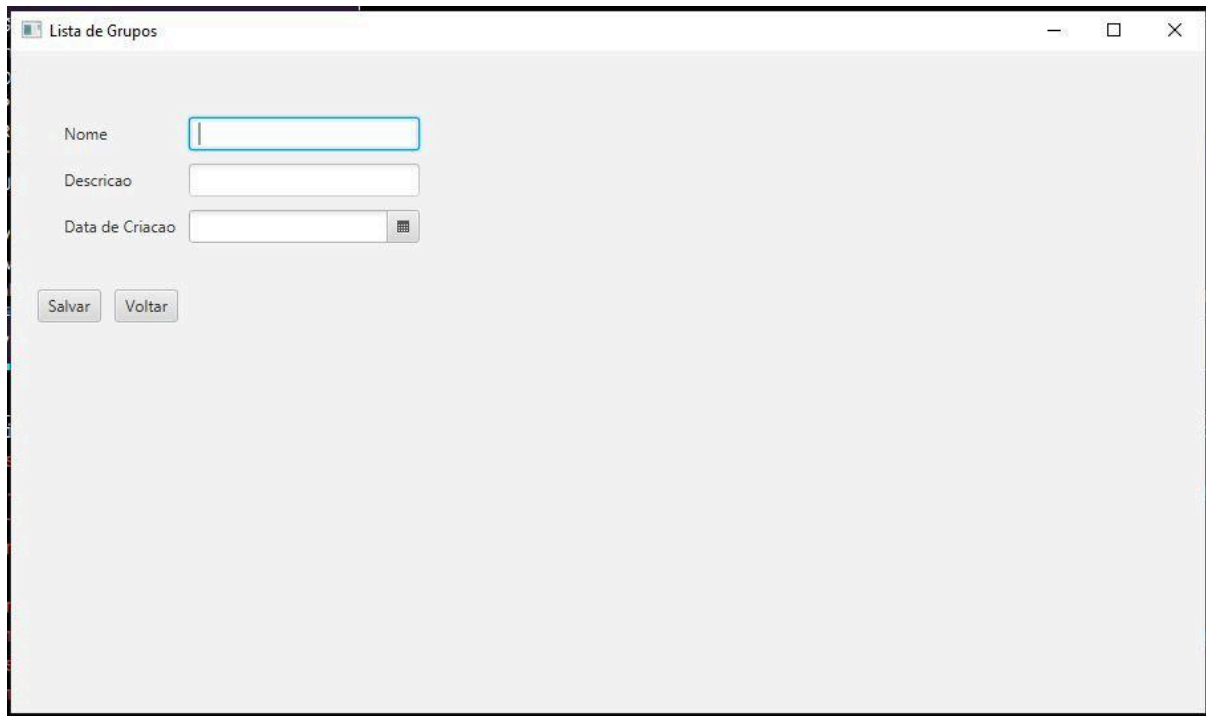
Aluno responsável - Josue Nonalaya

Descrição da Classe

A classe GrupoCreateView implementa a tela de cadastro de grupos. Ela permite que o usuário informe os dados necessários para criação de um novo grupo.

Componentes:

- Campo Nome: Nome do grupo
- Campo Descrição: Descrição do grupo
- Campo Data de Criação: Seleção da data de criação
- Botão Salvar: Cadastra o grupo
- Botão Voltar: Retorna à listagem



print da tela GrupoCreateView.java

Métodos

GrupoCreateView(Stage stage)

- Construtor que recebe a janela onde a tela será exibida.

mostrar()

- Monta a interface gráfica, captura os dados informados pelo usuário e realiza o cadastro do grupo quando o botão Salvar é acionado.

Classe: GrupoEditView

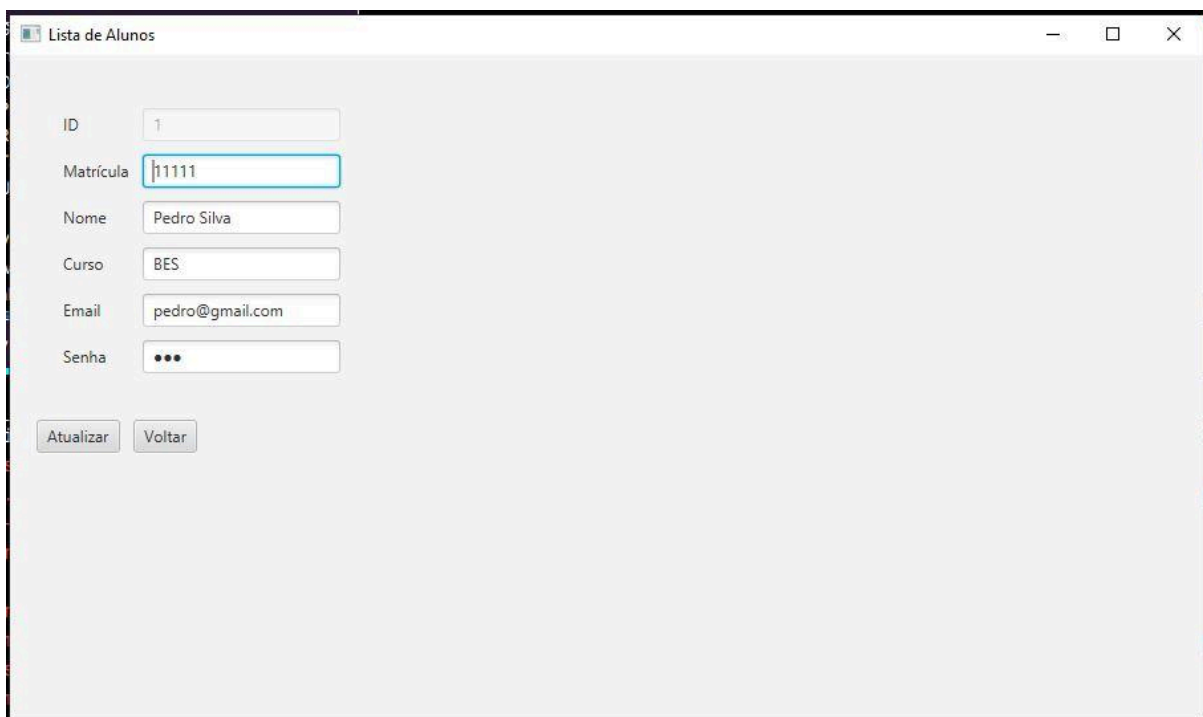
Aluno responsável - Josue Nonalaya

Descrição da Classe

A classe GrupoEditView implementa a tela de edição de grupos. Ela carrega os dados do grupo selecionado e permite a alteração dos dados.

Componentes:

- Campo ID: Exibe o identificador do grupo (somente leitura)
- Campo Nome: Permite alterar o nome do grupo
- Campo Descrição: Permite alterar a descrição
- Campo Data de Criação: Permite alterar a data de criação
- Botão Atualizar: Salva as alterações realizadas

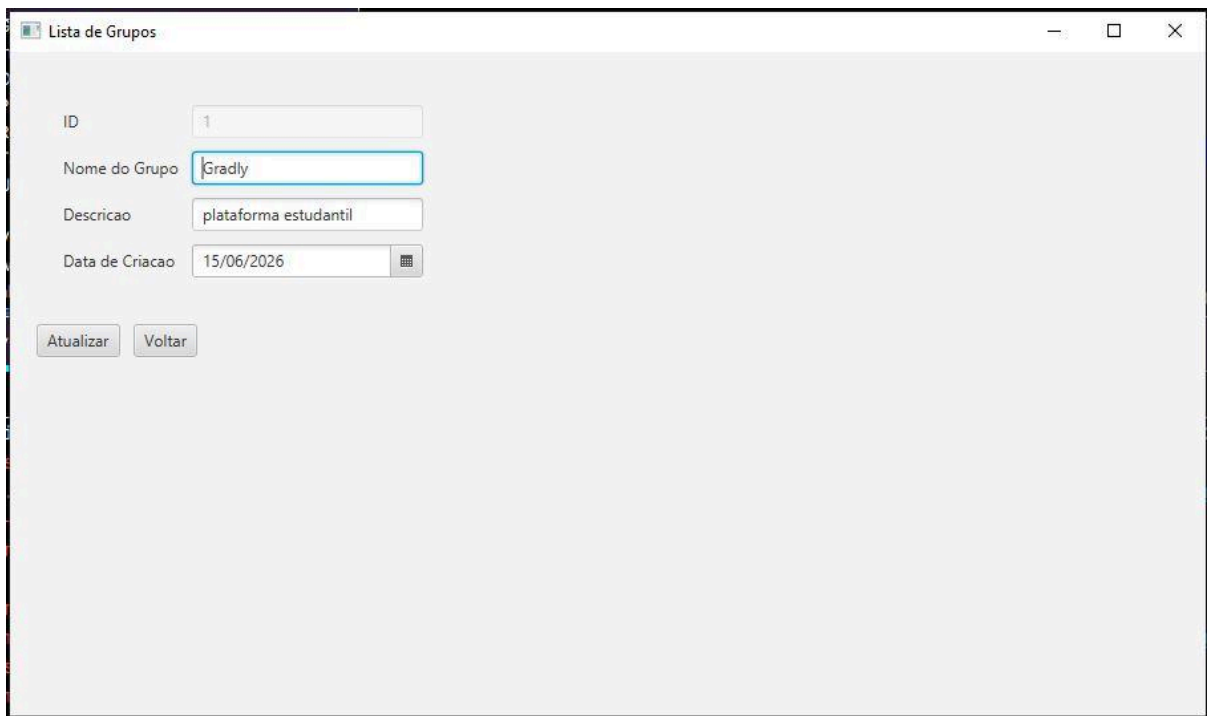


A imagem mostra uma interface gráfica de usuário para a edição de dados de um grupo. A janela principal é intitulada "Lista de Alunos" e possui uma barra de título com botões de minimizar, maximizar e fechar. O formulário contém os seguintes elementos:

- ID:** Campo de texto com o valor "1".
- Matrícula:** Campo de texto com o valor "11111", destacado por uma borda azul.
- Nome:** Campo de texto com o valor "Pedro Silva".
- Curso:** Campo de texto com o valor "BES".
- Email:** Campo de texto com o valor "pedro@gmail.com".
- Senha:** Campo de texto com pontos para ocultar o conteúdo.

Na base do formulário, há dois botões: "Atualizar" e "Voltar".

- Botão Voltar: Retorna à listagem



The screenshot shows a window titled "Lista de Grupos". Inside, there is a form with the following fields and values:

- ID: 1
- Nome do Grupo: Gradly
- Descrição: plataforma estudantil
- Data de Criação: 15/06/2026

At the bottom of the form, there are two buttons: "Atualizar" and "Voltar".

print da tela GrupoEditView.java

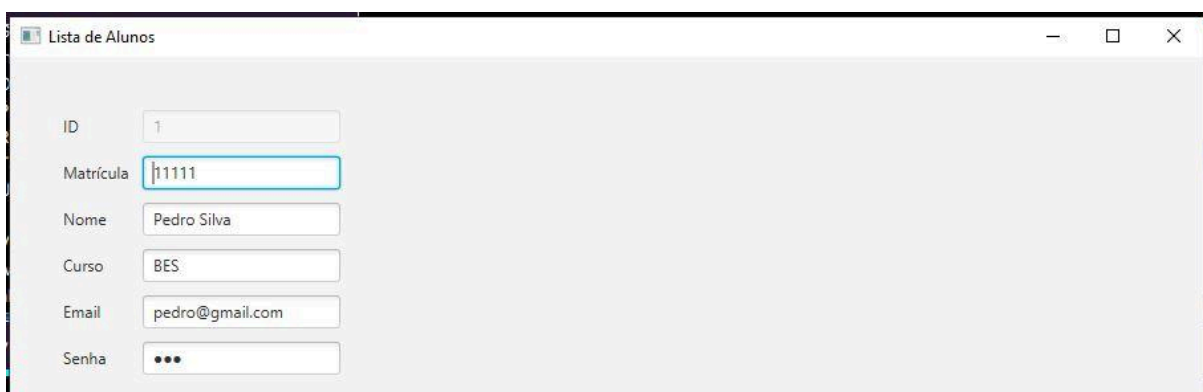
Métodos

GrupoEditView(Stage stage, Grupo grupo)

- Construtor que recebe a janela e o grupo que será editado.

mostrar()

- Monta a interface gráfica, preenche os campos com os dados atuais do grupo e realiza a atualização quando o botão Atualizar é acionado.



The screenshot shows a window titled "Lista de Alunos". Inside, there is a form with the following fields and values:

- ID: 1
- Matrícula: 11111
- Nome: Pedro Silva
- Curso: BES
- Email: pedro@gmail.com
- Senha: ...